IU International University of Applied Sciences

Project report

Course: Project: NLP (DLBAIPNLP01)

Degree programme: Bachelor of Science Applied Artificial Intelligence

Spam Detection with text classification: Effects of Data Size, Model Type and Text Type

Student: Arfaan Zameerahmed Sayyad

Matriculation number: 42308287

Tutor: Visieu Lac

Submission date: 26/10/2026

Table of contents

1 Introduction **1**

2 Theoretical Background **2**

3 Data and Preparation **3**

3.1 Datasets 3

3.2 Pre-Processing and Data Splitting 4

4 Implementation and Results **5**

4.1 Small-Data Stage: SMS Messages 5

4.2 Large-Data Stage: Enron Emails and Learning Curves 6

4.3 Generalization across Datasets 7

4.4 Neural Network versus Classical Models 8

5 Project Management **8**

6 Discussion **9**

7 Conclusion **10**

References **12**

List of Appendices **13**

Appendix A Additional Figures **14**

Appendix B Repository and Code Structure **15**

List of figures

Message length in words by class (log scale) 4

Confusion matrices of the three models on the SMS test set 6

Learning curves: test F1 against training size (shaded = ± 1 SD over 3 subsamples) 7

Class distribution of the two datasets 14

Words with the highest weights in the SMS Logistic Regression model 14

List of Table

Overview of the two datasets....3
Pre-processing variants on the SMS training data (5-fold CV F1, mean ± SD) 5
Test results on the SMS test set (1,010 messages) 6
Test results on the Enron test set (5,795 emails)....7
Cross-dataset test: F1 and ROC-AUC when training and test data differ 8
CNN + BiLSTM compared with the best classical model per dataset 8
Risks and measures 9
Python scripts of the repository (folder src/)....15

List of abbreviations

AUC	Area under the curve
BiLSTM	Bidirectional long short-term memory
CI	Confidence interval
CNN	Convolutional neural network
CPU	Central processing unit
CV	Cross-validation
FN	False negative
FP	False positive
HTML	Hypertext markup language
NB	Naive Bayes
NLP	Natural language processing
NLTK	Natural Language Toolkit
ROC	Receiver operating characteristic
SD	Standard deviation
SMS	Short message service
SVM	Support vector machine
TF-IDF	Term frequency–inverse document frequency
TN	True negative
TP	True positive

1 Introduction

Spam are the messages which are unwanted and it contains links that can be malicious and lead to financial loss or loss of personal information (Elakkiya et al., 2021, p. 9287). Detecting spam can be treated as a binary text classification task, each message is labelled one of two labels (spam or legitimate) and using supervised learning model a model can be trained using labelled dataset. This project was carried out for NLP (DLBAIPNLP01) course at IU International University of Applied Sciences, for task: Text Classification for Spam Detection, its used two public datasets (one of SMS messages and one of email).

The objective was to develop and evaluate a system that can classify message as spam or legitimate, also follow the steps required by the task such as data collection, pre-processing with stop-word removal, stemming or lemmatization, and term frequency-inverse document frequency (TF-IDF) encoding, training of supervised models, evaluation on a separate test set, and classification of new messages. There were Four main questions that guided the work, and that were as follow: which of three classical models and which pre-processing performed best, how the results changed from small to larger training sets, whether a model was trained on one dataset worked on the other and, whether a neural network improved on the classical models.

Messages were converted into numbers using TF-IDF, and other three classical models (Naive Bayes, Linear support vector machine (SVM) and Logistic Regression) were trained. Naive Bayes was chosen because it is popular in spam filters probably due to its really simple, has linear computational complexity and its accuracy (Metsis et al., 2006, p. 1). The SVM was chosen because it suits text, since the vectors have many dimensions but only a few non-zero values (Joachims, 1998, p. 140). Logistic Regression was added, as a third common linear model for comparison. A CNN+BiLSTM network was added because order of word is lost by the TF-IDF, but networks of LSTM type keep the sequential structure (Elakkiya et al., 2021, p. 9288). Also, traditional methods have sparse, high-dimensional features (Huan et al., 2022, p. 2108). All models were evaluated with metric and F1 score was main one, because spam is the minority class as only 13.4% of SMS message were spam and all the choices were based on 5-fold cross-validation on the training data and the test dataset was only used once that also for evaluation.

Two public datasets were used, first one is the copy of the SMS Spam Collection that had 5,572 messages and 13.4% of them were spam (Almeida & Hidalgo, 2011), and another one was Enron-Spam, it contained legitimate emails from six Enron employees and had spam from other sources (Metsis et al., 2006, p. 4). Both datasets are different in terms of text type and size as SMS message are short compared to longer emails, but these datasets allow both step by step increase of data size and also enables a cross-dataset test.

As Chapter 2 explains about the models, the theory of TF-IDF and the metrics, and the data and methods is explained in the Chapter 3. Chapter 4 shows the results of the experiments, Chapter 5 describes how the project was managed, Chapter 6 discusses the results and also their limitations and Chapter 7 is the conclusions of the project. The code, the result collected from the experiments and this report are uploaded on GitHub at https://github.com/Arfaan-S/spam-detection-nlp (the scripts are listed in Appendix B).

2 Theoretical Background

Each message must be first converted into numbers because classifiers cannot read the text. Each word is given a weight using Term frequency-inverse document frequency (TF-IDF), if the word occurs more often in a message, higher weight is assign to it ((term frequency), but if the words occurs in multiple messages, lower weight is assigned (inverse document frequency) (Jooachims, 1998, p. 138). Therefore, each message is represented as a long vector in which most values are zero (Joachims, 1998, p. 140).

Naive Bayes (NB) estimates the probability for the message for being spam using the words it contains (Metsis et al., 2006, p. 2) NB assumes that the words are independent from each other in same class, that is why it is called naive. A support vector machine (SVM) uses hyperplane or line that leaves the widest possible margin to separate the classes. SVMs suit text because there are multiple dimensions in vectors but only few are non-zero values (Joachims, 1998, p. 140). Logistic Regression combines the values of TF-IDF with weight it has learned and the results are transformed into probability between 0 and 1. With grid search setting of each model was tuned (Chapter 3), such as alpha was used for NB, C for SVM and also for Logistic Regression. To control how closely the models fits the training data, value C is used.

A neural network learns its own features from the text, rather than just using fixed weights of TF-IDF. A Convolutional neural network (CNN) is good at detecting patterns of short words, that is why it can extract local features, but the dependencies over longer distance cannot be captured by it (Huan et al., 2022, p. 2109). To deal with this problem a bidirectional long short-term memory network (BiLSTM) manages to read both forward and backward text (Elakkiya et al., 2021, p. 9288). This help to keep information for longer distance and with the help of gates its able to decide what to remember and what to forget (Elakkiya et al., 2021, p. 9292). By combining both local and global features can be achieved (Huan et al., 2022, p. 2109), for this project it is combined them in one network (CNN+BiLSTM).

A classifier's results are counted in a confusion matrix with true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN). Positive class for spam and FP is for legitimate message that is flagged as spam. (Metsis et al., 2006, p. 5). Precision (P) is the share of flagged messages that are really spam and given by TP/(TP + FP), recall (R) is the share of all spam that

was found, is given by TP/(TP + FN), and F1 score is their harmonic mean, (2 x P x R) / (P + R) (Elakkiya et al., 2021, p. 9296). If the data is unbalanced then the accuracy alone can mislead, as in SMS, 13.4% was spam, so the model that labels everything legitimate would be 86.6% accurate, therefore F1 score was the main metric. The receiver operating characteristic, area under the curve (ROC-AUC), measures how well spam is ranked above legitimate messages across all thresholds, 0.5 means random ranking and 1 means perfect ranking.

3 Data and Preparation

3.1 Datasets

Two publicly available datasets were used for this project, and those are the SMS Spam Collection and Enron-Spam. These datasets contain raw text, so technique like stop word removal and stemming can be applied. Both are different in terms of size and text type, such as SMS is short whereas email is longer, but this allows to start with small dataset and continue with the larger one and to test whether a model trained on one dataset also works on the other.

The copy of the SMS Spam Collection contains 5,572 messages, of which 747 of them are spams, which makes it 13.4% of whole dataset. More than half of spams comes from a UK forum where users report the message as spam. Actual legitimate messages are mainly from a university SMS corpus. This dataset is licensed under CC by 4.0 (Almeida & Hidalgo, 2011).

The Enron-Spam corpus consists of 33,716 emails, of which 16,545 were legitimate email and 17,171 were spam (MWiechmann, n.d.). 51 empty texts were removed and 33,665 remained. The totals of the copy used equal the sum of the six datasets of the original paper (Metsis et al., 2006, p. 5). This dataset was created by Metsis et al. (2006, p. 4), with the email of 6 Enron employees and rest were separately collected spam from four different sources.

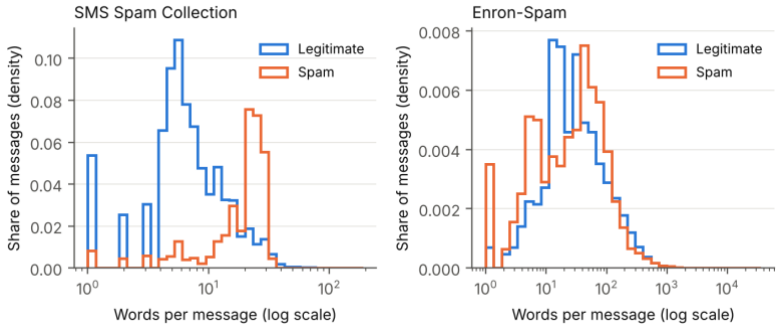
Tab. 1 shows that the datasets differ, only 13.4% of SMS are spam which makes it imbalanced (Fig. 4, Appendix A), on other hand 50.9% of email are spam, so it's nearly balanced. Length of SMS message is short, its median is 10 words for legitimate and 24 words for spam messages, and length of email are long, it had median of 120 words for legitimate and 97 words for spam emails. But in the SMS data, spam is typically longer than legitimate SMS, whereas length distributions of the emails strongly overlap (Fig. 1).

Tab. 1: Overview of the two datasets

Dataset	Messages	Spam share (%)	Exact duplicates (%)	Train	Test	Median words (legitimate / spam)
SMS Spam Collection	5,572	13.4	403 (7.2)	4,040	1,010	10 / 24
Enron-Spam	33,665	50.9	3,203 (9.5)	23,179	5,795	120 / 97

Source: Own representation.

Fig. 1 Message length in words by class (log scale)



Source: Own representation.

3.2 Pre-Processing and Data Splitting

Before any model was trained, duplicate messages were removed, as shown in Tab. 1, 7.2% of SMS data which is 403 were exact duplicates and in emails datasets 9.5% which is 3,203 emails were duplicates. None of the text carried both labels. This was done before the split because the identical text from the training and test dataset will cause problem called data leakage, in which test score is inflated.

Each text was converted to lower case, tags of HTML were removed, placeholder such as urltoken, mailtoken, were used replacing web addresses, email addresses, money amounts and numbers. The placeholders were used because there are often links, price and phone number in spam as their exact value is not useful, but their appearance is more informative. There effect of this technique is analysed in Section 4.1. Stop words were removed using list of NLTK English that has 198 words. Words were transformed into their base form, either by using lemmaatization or by stemming. Lemmatization replaces the word by its dictionary form, e.g. “days” becomes “day”, this is done by using the lexical database WordNet (Miller, 1995, p. 39). On other hand Stemming cuts, the ending of the word using Porter algorithm (porter, 1980/2006, p. 211), e.g. “customer” becomes “custom”. Both versions were compared in Section 4.1. Stemming was used for the classical models and the lemmatized text for the neural network.

Further check showed that after cleaning that data, 28 SMS and 204 emails were identical in training and test data. Therefore, all 110 SMS and 1,487 emails were identical after cleaning (near-duplicates), therefore they were removed, in addition 9 SMS and 1 email, became empty after cleaning and were removed. After this step, the dataset contained 5,050 SMS and 28,974 emails, also training and test data shared no identical text.

The remaining data were split into 80% for training and 20% for testing data. For both parts to have the same share of spam, data was stratified and random seed value of 42 allows the split reproducible. As show in Tab. 1 SMS had 4,040 for training and 1,010 for testing, for Enron 23,179 for training and 5,795 for test. The test data was not used for any choice (pre-processing, tuning or model selection). All the choices, such as pre-processing and model settings were made using 5-

fold cross validation on the training data. The test was converted to number using TF-IDF, it was fitted on the training data only. Therefore, with the single words as features this gives, 2,724 features for SMS and 56,051 for Enron and this method is explained in Chapter 2.

4 Implementation and Results

4.1 Small-Data Stage: SMS Messages

Before the models were tuned, five pre-processing variants were compared by 5-fold cross-validation on the training data (F1 score, default model settings): A (lower case only), B (A plus placeholders), C (B plus stop-word removal), D (C plus lemmatization) and E (C plus stemming). Tab. 2 indicates that the introduction of placeholders from A to B improved Logistic Regression the most (from 0.815 to 0.889), but had little effect on Naive Bayes. Stop-word removal improved Naive Bayes (from 0.830 to 0.862) but other two models are lowered. Stemming was chosen because it had the highest average over the three models, however B score for Linear SVM and Logistic Regression was higher. The difference to lemmatization was smaller than the fold-to-fold variation, thus this choice should not be over-interpreted.

For the chosen variant, the parameters of each model were tuned by grid search (testing all combinations from a list of settings) with 5-fold cross-validation on the training data, which is why the cross-validation F1 in Tab. 3 is higher than in Tab. 2 (default settings). From Tab. 3 one can see that all models achieved an accuracy above 0.98, the main difference being the recall. Logistic Regression had the highest test F1 of 0.943 and was chosen because it had the highest cross-validation F1, thus test set was not used for the ranking. However, the 95% confidence intervals overlapped and the McNemar test (comparing the errors of two models) yielded p-values 0.15-0.77. The test set only contained 117 spam messages, thus the ranking should be interpreted with caution.

Logistic Regression made 13 errors on the 1,010 test messages (Fig. 2): 3 legitimate messages were falsely flagged as spam and 10 spam messages were missed. Two of the three falsely flagged messages were very short everyday messages. Some of the missed spam messages had the style of personal messages, for example, "How come it takes so little time for a child who is afraid of the dark…", which shows that spam without the typical advertising words is difficult to detect. The spam features with the highest weights include the placeholders urltoken and numtoken (Fig. 5, Appendix A).

Tab. 2: Pre-processing variants on the SMS training data (5-fold CV F1, mean ± SD)

Pre-processing variant	Naive Bayes	Linear SVM	Logistic Regression
A lowercase only	0.827 ± 0.041	0.922 ± 0.018	0.815 ± 0.029
B + placeholders (url, money, number)	0.830 ± 0.046	0.940 ± 0.021	0.889 ± 0.028
C + stop words removed	0.862 ± 0.053	0.931 ± 0.025	0.864 ± 0.032
D + lemmatized	0.868 ± 0.047	0.932 ± 0.028	0.869 ± 0.035
E + stemmed (instead of D)	0.876 ± 0.045	0.933 ± 0.028	0.874 ± 0.036

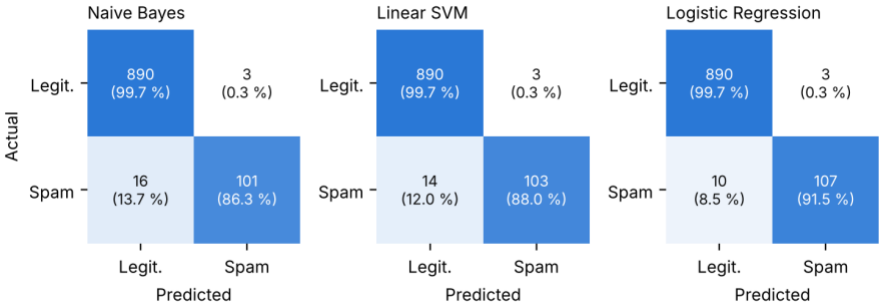
Source: Own representation.

Tab. 3: Test results on the SMS test set (1,010 messages)

Model	CV F1	Accuracy	Precision	Recall	Test F1 [95 % CI]
Naive Bayes	0.934	0.981	0.971	0.863	0.914 [0.873; 0.950]
Linear SVM	0.935	0.983	0.972	0.880	0.924 [0.884; 0.957]
Logistic Regression	0.938	0.987	0.973	0.915	0.943 [0.910; 0.971]

Source: Own representation.

Fig. 2 Confusion matrices of the three models on the SMS test set



Source: Own representation.

4.2 Large-Data Stage: Enron Emails and Learning Curves

The same procedure was followed for the Enron emails: the variant of the pre-processing pipeline that was selected for the SMS data was used without further changes, and the same type of grid search with 5-fold cross-validation, using a smaller grid of settings. Tab. 4 shows that all three models achieved an F1 score above 0.99. Linear SVM had the highest test F1 (0.994) and also the highest cross-validation F1 (0.9912 vs. 0.9905 and 0.9902), so it was selected as the final model for Enron. The difference in F1 scores was at most 0.004, and the confidence intervals partly overlapped.

The F1 scores of 0.990 to 0.994 on the Enron dataset were higher than those on the SMS dataset (0.914 to 0.943). This was likely due to difference in the text type, such as the longer emails and more balanced classes. The confidence intervals were also narrower (at most 0.005 vs. 0.06 to 0.08 for SMS), since the test set had 2,899 spam emails compared to 117 spam messages.

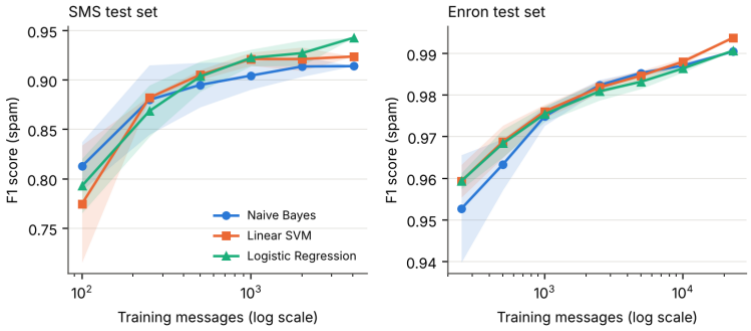
Fig. 3 illustrates the F1 score as a function of the training set size, which increased from 100 to 4,040 messages for SMS and from 250 to 23,179 emails. Sizes that are below the full training set were drawn as 3 random subsamples each, and the F1 score was evaluated on the same test set in each case. For SMS, most of the improvement came in the first 1,000 messages: Linear SVM had a F1 score of 0.775 for 100 messages, 0.921 for 1,000 and 0.924 for 4,040. The curves for the emails increased more gradually and did not flatten at the full size (0.959 for 250 emails, 0.988 for 10,000 and 0.994 for 23,179 emails), so further increasing the amount of training data is likely to continue improving the results slightly. At the similar training size, Linear SVM managed to reach 0.985 with 5,000 emails against 0.924 with 4,040 SMS messages, therefore training size alone does not explain the higher Enron scores.

Tab. 4: Test results on the Enron test set (5,795 emails)

Model	CV F1	Accuracy	Precision	Recall	Test F1 [95 % CI]
Naive Bayes	0.9905	0.991	0.992	0.989	0.990 [0.988; 0.993]
Linear SVM	0.9912	0.994	0.992	0.996	0.994 [0.992; 0.996]
Logistic Regression	0.9902	0.991	0.988	0.993	0.991 [0.988; 0.993]

Source: Own representation.

Fig. 3 Learning curves: test F1 against training size (shaded = ± 1 SD over 3 subsamples)



Source: Own representation.

4.3 Generalization across Datasets

To test generalization, models that was trained on one dataset was evaluated on the other dataset (Tab. 5). Linear SVM trained on SMS achieved F1 score of 0.501 on the Enron test set, compared to SMS test set (0.924). Linear SVM trained on Enron reached 0.244 on the SMS test set compare with Enron test set (0.994), but ROC-AUC decreased from 0.991 to 0.592 in first case, and from 0.999 to 0.688 in the second.

The two directions failed in opposite ways. Linear SVM trained on SMS dataset missed 1,686 out of 2,899 spam emails. On other hand, Linear SVM trained on emails, flagged 673 out of 893 legitimate SMS message as spam, whereas it only missed seven of the 117 spam messages. This was likely due to the different vocabulary and message length, SMS are shorter and emails are longer (Tab. 1).

Training on both datasets together kept the Enron result almost unchanged as F1 score of Linear SVM barely changed (0.994 to 0.984), but the score of SMS went from 0.924 to 0.807. The SMS F1 decreased because 44 of legitimate message were flagged as spam (against 3 before), whereas ROC-AUC remained high at 0.988, this indicates that spam was ranked higher than legitimate messages.

Because the legitimate Enron emails came from Enron employee and the spam came from other sources (Metsis et al., 2006, p.4), the model could have learned words of company as a shortcut. So by removing five such words (e.g., “Enron”) changed the Enron F1 by max 0.003, as Linear SVM only changed from 0.994 to 0.991, and F1 of SMS stayed between 0.24 and 0.26. Therefore, the poor transfer is not explained by company specific words.

Tab. 5: Cross-dataset test: F1 and ROC-AUC when training and test data differ

Trained on	Tested on	Naive Bayes F1	Linear SVM F1	Logistic Regression F1	Linear SVM ROC-AUC
SMS	SMS	0.914	0.924	0.943	0.991
SMS	Enron	0.264	0.501	0.502	0.592
Enron	Enron	0.990	0.994	0.991	0.999
Enron	SMS	0.257	0.244	0.232	0.688
SMS + Enron	SMS	0.575	0.807	0.791	0.988
SMS + Enron	Enron	0.989	0.984	0.985	0.999

Source: Own representation.

4.4 Neural Network versus Classical Models

Tab. 6 compares the CNN+BiLSTM network with the best classical model on each dataset. The network results are the mean ± standard deviation over three seeds. On SMS, the network achieved F1 score of 0.912 ± 0.024, compared to Logistic Regression (0.943). On Enron, the network achieved F1 score of 0.988 ± 0.002, compared to Linear SVM (0.994). In the end, the network did not outperform the classical model on either dataset.

On SMS, the lower F1 score resulted mainly from recall. The network's recall, which was 0.869 for network and 0.915 for Logistic Regression. Precision was close (0.959 and 0.973). The network's F1 score varied between seeds (0.887 to 0.934), and even the best seed was below 0.943.

The network also required much more training time; 20.9 s compared to 0.03 s on SMS and 526.9 s against 1.11 s on Enron. In addition, it had about 206,000 parameters on SMS and 3.8 million on Enron. The SMS result was lowered likely due to overfitting, as there are 4,040 messages in the training set (Elakkiya et al., 2021, p. 9295). For this task, the classical models were preferable because it achieved higher F1 scores and It was much faster. This applies to the reduced network used here, which had no pretrained embeddings and no attention.

Tab. 6: CNN+BiLSTM compared with the best classical model per dataset

Dataset	Model	Test F1	Recall	ROC-AUC	Training time (s)
SMS	Logistic Regression (best classical)	0.943	0.915	0.991	0.03
SMS	CNN+BiLSTM (3 seeds, mean ± SD)	0.912 ± 0.024	0.869	0.986	20.9
Enron	Linear SVM (best classical)	0.994	0.996	0.999	1.11
Enron	CNN+BiLSTM (3 seeds, mean ± SD)	0.988 ± 0.002	0.993	0.999	526.9

Source: Own representation.

5 Project Management

The project was done by one student, who planned the whole work, wrote the code, ran experiments on it and wrote the report about it. The code was written in Python and using libraries like scikit-learn 1.9.1, NLTK and PyTorch, and both the code and report are uploaded to GitHub. Also, the dataset was free and publicly available, so no personal data were used. Everything was run only on CPU, but only neural network is slow.

The order required by the task was followed for this project, from a smaller dataset to larger once. On 12 October, the data was collected, analysed and pre-processed. On 13 October, the models were developed on SMS messages which has total 5,050 messages after cleaning (4,040 for training and 1,010 for test), along with Enron data (28,974 emails) and different dataset was used for testing, the neural network and prediction demo was also tested. From 14 to 15 of October Enron copy was checked using the original paper and the sources were verified.

Tab. 7 lists the four main risks, the measures taken and what happened.

Tab. 7: Risks and measures

Risk	Measure	What happened
Leakage: the same text in training and test	Exact duplicates removed before the split and processed texts compared	28 (SMS) and 204 (Enron) shared texts found, and near-duplicates removed, so 0 overlap
Optimistic test scores	All choices by 5-fold cross-validation on training data and the test set was used only for the final evaluation	Logistic Regression was chosen by cross-validation F1, not by test F1
Limited computing power	Smaller grid for Enron, reduced network (12 epochs, 3 seeds)	Best Enron values lay at the grid edge (Chapter 6)
Different pre-processing at prediction time	Demo script with new example messages	Demo with lemmatized text wrongly flagged emails, fixed with stemming

Source: Own representation.

The plan was adjusted twice during the work. At first, because the leakage was checked and it showed that identical texts appeared in both training and testing data even after cleaning, so these near duplicates were removed before the split. Second, the best Logistic Regression was laying right at the upper edge of the first grid that had value C = 100, so the gird was increased to value of 1000, and the best value remained at 100. At the start the Enron grid was smaller, because of the larger dataset and it was not extended (see Chapter 6).

6 Discussion

The goal was to classify the messages as spam or legitimate, and it was achieved. The best models on test dataset reached F1 scores of 0.943 for Logistic Regression on SMS and 0.994 for Linear SVM on Enron. The task required to start with a small dataset and then use larger ones, for that learning curve ran from 100 to 4,040 SMS messages and from 250 to 23,179 emails, due to this F1 score rose but with smaller gain with larger sizes (see Fig. 3). In the prediction demo, all of the eight example messages were classified correctly but they are not enough to check the quality just show how system is used.

All three classical models performed similarly, for SMS all three McNemar tests gave p-value above 0.05 (0.77, 0.15, 0.22), so no difference was significant and for Enron F1 score was between 0.990 and 0.994. The best F1 score was achieved by the linear SVM model on Enron, but SVMs suit text because the vectors have multiple dimensions and a few non-zero values (Joachims, 1998, p. 140), but Logistic Regression performed best on SMS and the differences were small, so there are no clear advantage of SVM usage. The cross-dataset test showed that the models learned the signals

only from one dataset, not of spam in general that's why F1 score was between 0.23 and 0.50 and ROC-AUC fell between 0.59 and 0.73. Company specific words don't explain this result, because by removing them from dataset, changed the Enron F1 score by less than 0.003 and the cross-dataset F1 score changed by about 0.01 or less.

The SMS test dataset only had 117 spam messages, so one message changes recall by about 0.9 percentage points, and the confidence intervals were wide, for example 0.910 to 0.971 for Logistic Regression, and the small difference between the models are not reliable. The Enron grid was smaller than the SMS grid and the best value of the Linear SVM (C = 3) and Naive Bayes (alpha = 0.1) lay at its edge, but slightly better settings may exist. The lemmatizer was used without part-of-speech tags, so all the words were treated as nouns. The Network used the lemmatized text and for the classical models stemmed text was used, but this difference was not tested. The Enron emails which were legitimate came from only six employees of the company (Metsis et al., 2006, p. 4), so the results may not hold for other mailboxes. A reduced version of network was used, which was without pretrained embeddings and no attention (see Section 4.4).

In the future, the Enron grid search could be repeated with the wider range and use part-of-speech tags and test attention (Elakkiya et al., 2021, p. 9287). The computing time for classical models was far less (Tab. 6). The datasets were published in 2006 and 2011, so the spam wording changed over time and the models would require retraining on new messages before real use.

7 Conclusion

Overall, project showed that a classical model that used TF-IDF identify spam well when the model was trained on same type of data as test dataset, so the best test F1 score for Logistic Regression (based on SMS) was 0.943 and for Linear SVM (based on Enron) was 0.994. All the required task by this project were carried out such as data collection, pre-processing, training, evaluation on separate test set.

First, the choice of model was not that important as the three classical models F1 scores of SMS achieved 0.914 to 0.943 and 0.990 to 0.994 on Enron differences on SMS were not significant. Among the pre-processing steps, the one that improved the most was replacing the URLs, money amounts and numbers with placeholders, for example the cross-validation F1 of Logistic Regression rose from 0.815 to 0.889, whereas stemming and lemmatization step had small impact on improvement. More training data helped, as F1 score rose, it was 0.8 with 100 SMS messages to above 0.91 with 4,040 and from about 0.95 with 250 emails to about 0.99 with 23,179, but the gains were smaller with more data. A model trained that was trained on one dataset, struggle on the other dataset, as result F1 score was between 0.23 and 0.50, this suggests that the model trained learns only signals of one dataset, but a model trained on both datasets kept its Enron score but also perform bad on SMS than the SMS only model. The CNN+BiLSTM network did not improve on

classical models, as F1 reached only 0.912 ± 0.024 against 0.943 on SMS and 0.988 ± 0.002 against 0.994 on Enron, also the computing time needed was far more (Tab. 6).

For this task, a good choice would be a simple classic model because it achieved the best scores and its training time is fraction of time taken by network. However, for the model to perform good, its important that it's only used on type to text that it was trained on, because the F1 score of cross-dataset was between 0.23 and 0.50 and for new type of text needs retraining. These conclusions hold within the limits as described before in Chapter 6, such as the small SMS test set that contain 117 spam messages, and the smaller Enron grid.

Future studies, models can be tested on newer spam data and on more than just two datasets, so that it can help to check to figure out if the failure across datasets also occurs there. The project also helped to gain practical experience in a full machine learning workflow such as data cleaning, evaluation with cross-validation and statistical tests and data leak. This experience can be used for further studies or later work, for example news analysis and evaluating its impact of stock prices.

References

Almeida, T., & Hidalgo, J. (2011). *SMS spam collection* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: Textual content based deep learning framework for social spam detection using conjoint attention mechanism. *Journal of Ambient Intelligence and Humanized Computing, 12*(10), 9287–9302. https://doi.org/10.1007/s12652-020-02640-5

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. *Connection Science, 34*(1), 2108–2124. https://doi.org/10.1080/09540091.2022.2098926

Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. In C. Nédellec & C. Rouveirol (Eds.), *Machine Learning: ECML-98* (pp. 137–142). Springer. https://doi.org/10.1007/BFb0026683

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with Naive Bayes – Which Naive Bayes? *Proceedings of the 3rd Conference on Email and Anti-Spam (CEAS 2006)*. Mountain View, CA, USA. https://www.researchgate.net/publication/221650814_Spam_Filtering_with_Naive_Bayes_-_Which_Naive_Bayes

Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM, 38*(11), 39–41. https://doi.org/10.1145/219717.219748

MWiechmann. (n.d.). *enron_spam_data* [Data set]. GitHub. https://github.com/MWiechmann/enron_spam_data

Porter, M. F. (2006). An algorithm for suffix stripping. *Program: Electronic Library and Information Systems, 40*(3), 211–218. https://doi.org/10.1108/00330330610681286 (Original work published 1980)

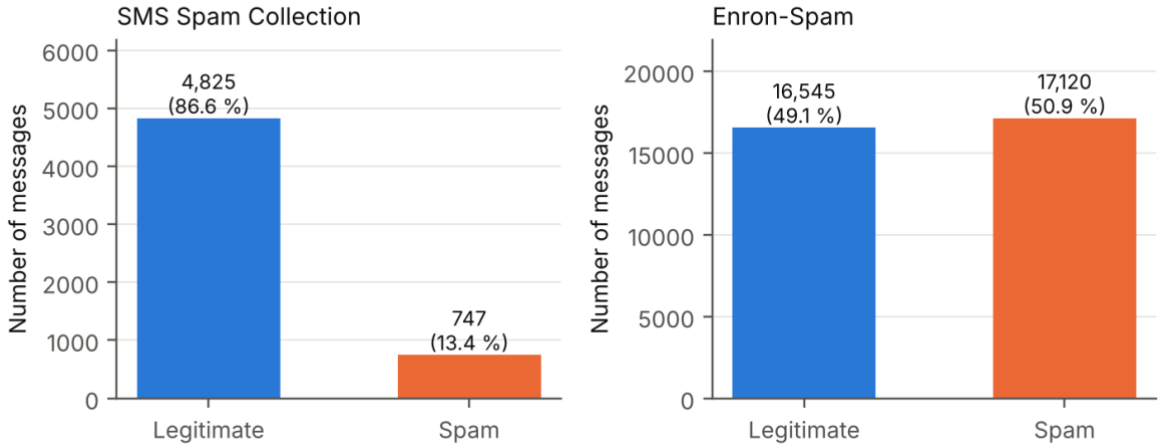
List of Appendices

Appendix A Additional Figures

Appendix B Repository and Code Structure

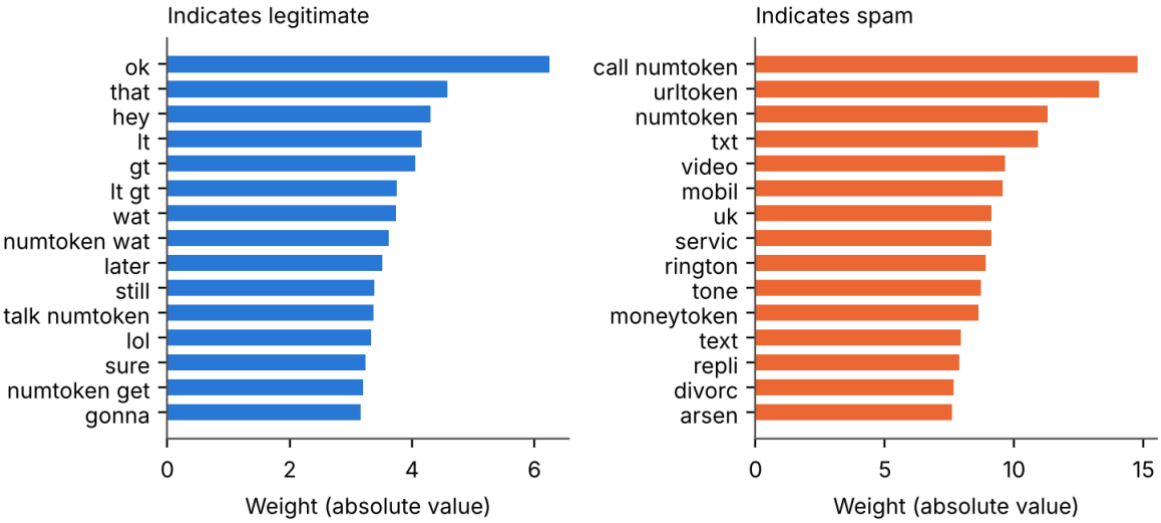
Appendix A Additional Figures

Fig. 4 Class distribution of the two datasets



Source: Own representation.

Fig. 5 Words with the highest weights in the SMS Logistic Regression model



Source: Own representation.

Appendix B Repository and Code Structure

Code, results and this report: https://github.com/Arfaan-S/spam-detection-nlp

Tab. 8: Python scripts of the repository (folder src/)

Script	Purpose
download_data.py	Downloads the two public datasets
data_loader.py	Loads both datasets into one common format
explore_data.py	Dataset statistics and exploratory figures
preprocessing.py	Cleaning, tokenization, stop words, lemmatization, stemming
features.py	TF-IDF encoding
prepare_data.py	Duplicate removal and stratified train/test split
evaluation.py	Accuracy, precision, recall, F1, bootstrap confidence interval
step5_sms_models.py	SMS: pre-processing study, tuning, test evaluation, McNemar test, error analysis
step6_scaling_generalization.py	Enron models, learning curves, cross-dataset test, bias check
step6b_neural_network.py	CNN+BiLSTM neural network
step7_predict_demo.py	Classifies new messages with the saved models

Source: Own representation.